МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КУБГУ»)

Кафедра вычислительных технологий

Отчёт

по лабораторной работе №3

по дисциплине «Методы разработки трансляторов»

на тему: «Перевод ОПЗ исходного

выражения в текст на выходном языке»

Работу выполнил

студент группы

ФИ 31/2

Костров А.А.

_____

(подпись)

Преподаватель:

Усов П.Е.

_____

(подпись)

Краснодар
2026

Цель работы

Изучить принципы генерации кода на основе обратной польской записи (ОПЗ) и реализовать программу-транслятор, преобразующую выражения языка R в код на языке Python.

Теоретическая часть

Генерация выходного кода является завершающим этапом работы компилятора. В данной работе в качестве промежуточного представления используется обратная польская запись (ОПЗ), полученная на предыдущем этапе.

Транслятор обрабатывает ОПЗ слева направо. При чтении операндов они помещаются в стек. При поступлении оператора или ключевой конструкции из стека извлекается необходимое количество аргументов, после чего генерируется соответствующий фрагмент кода на выходном языке.

В данной работе выходным языком является Python. Это накладывает следующие ограничения и особенности:

Использование синтаксических отступов (indentation) вместо фигурных скобок {} для блоков кода.

Замена векторных операций R на скалярные или поэлементные Python.

Эмуляция функции ifelse() через тернарный оператор Python x if cond else y.

Преобразование индексации массивов с 1-based (R) на 0-based (Python).

Описание программы

PythonCodeGenerator: реализует классический подход с интерпретацией ОПЗ и эмуляцией стека временных переменных, максимально приближенный к описанию в методическом пособии (использование внутренних переменных _tN и меток).

Программа интегрирована в общий GUI-интерфейс с лексическим анализатором и конвертером ОПЗ.

Основные возможности программы

Перевод арифметических и логических выражений.
Обработка присваиваний (<-, =, :=).
Генерация условных конструкций (if, else).
Генерация циклов (while).
Преобразование функций ifelse().
Эмуляция доступа к элементам массивов (операция АЭМ).
Обработка объявления функций (процедур).

Алгоритм генерации кода (подробно)

В основе работы PythonCodeGenerator лежит интерпретация команд ОПЗ, представленных в задании (УПЛ, БП, Ф, АЭМ и т.д.).

Основные шаги алгоритма PythonCodeGenerator:

Инициализация: Счетчики временных переменных (temp_counter), уровень отступа (indent_level), пустой стек python_stack.

Сканирование ОПЗ: Входная строка ОПЗ разбивается на токены и обрабатывается слева направо.

Обработка операндов: Если токен не является специальной операцией ОПЗ, он конвертируется в Python-совместимое значение (TRUE -> True, NA -> None) и помещается в стек.

Обработка операций: Для бинарных операторов (+, -, *, ^ и т.д.) из стека извлекаются два операнда, создается новая временная переменная (_tN), генерируется строка присваивания, которая добавляется в выходной листинг.

Обработка переходов:

УПЛ (Условный переход): Генерирует строку if cond: и увеличивает отступ.

БП (Безусловный переход): Генерирует else:.

Обработка массивов (АЭМ): Извлекает имя массива и индексы. Выполняет трансформацию индексов i -> (i - 1) для совместимости с Python.

Обработка процедур (НП/КП): Генерирует заголовок def name(): и увеличивает отступ.

Разбор реализации в файле output.py

Класс PythonCodeGenerator

Реализует эмуляцию вычислительной машины со стеком.

Основные структуры:

python_stack: Стек операндов (имена переменных, временные регистры).

output_lines: Список строк генерируемого Python-кода.

indent_level: Текущий уровень отступа.

Реализация ключевых семантик:

Бинарные операции:

```
def _process_binary_op(self, op):
    right = self.python_stack.pop()
    left = self.python_stack.pop()
    py_op = {'^': '**', '%%': '%', '&': 'and'}.get(op, op)
    self.temp_counter += 1
    temp_var = f"_t{self.temp_counter}"
    self._add_line(f"{temp_var} = ({left} {py_op} {right})")
    self.python_stack.append(temp_var)
```

Индексация массивов (АЭМ):

Учитывает особенность R, где индексы начинаются с 1.

```
def _process_array_access(self, tokens, pos):
    # ...
    py_indices = []
    for idx in indices:
        if idx.isdigit():
            py_indices.append(str(int(idx) - 1))
        else:
            py_indices.append(f"({idx} - 1)")
    access_str = arr_name + ".join(f'[{i}]' for i in py_indices)
    self.python_stack.append(access_str)
```

Управление блоками и отступами:

Так как Python требует строгих отступов, генератор хранит счетчик indent_level.

```
def _process_proc_begin(self):
    name = self.python_stack.pop()
    self._add_line(f"def {name}():")
    self.indent_level += 1
```

Пример работы

Исходный код на R:

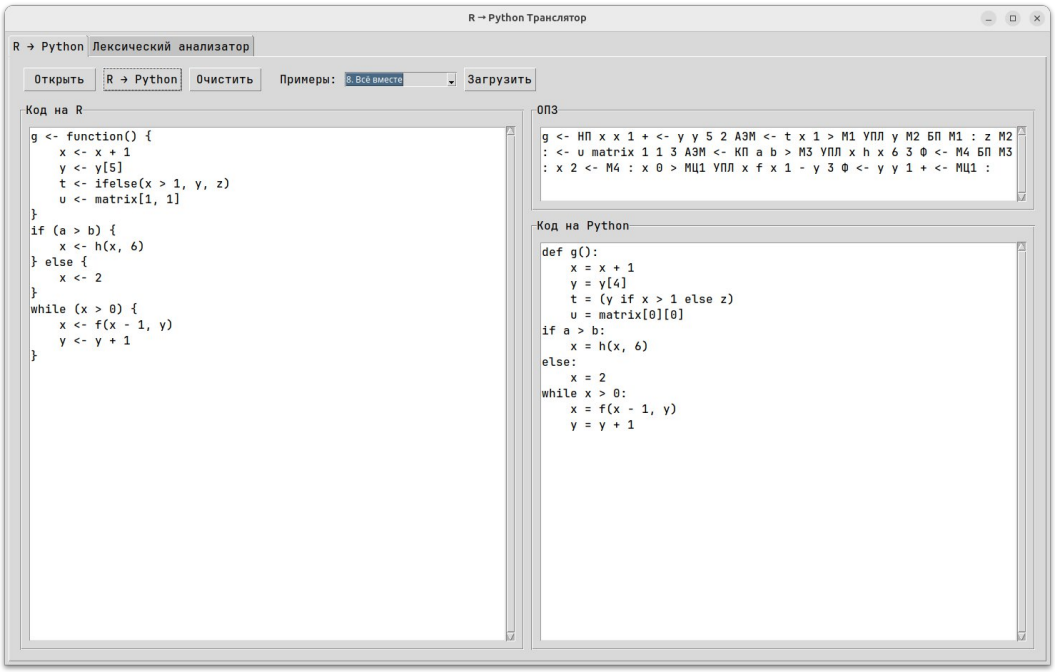
```
if (x > 10) {
    x <- x - 1
} else {
    x <- 3
}
y <- ifelse(a > b, a + 1, b - 1)
```

Результат генерации (Python):

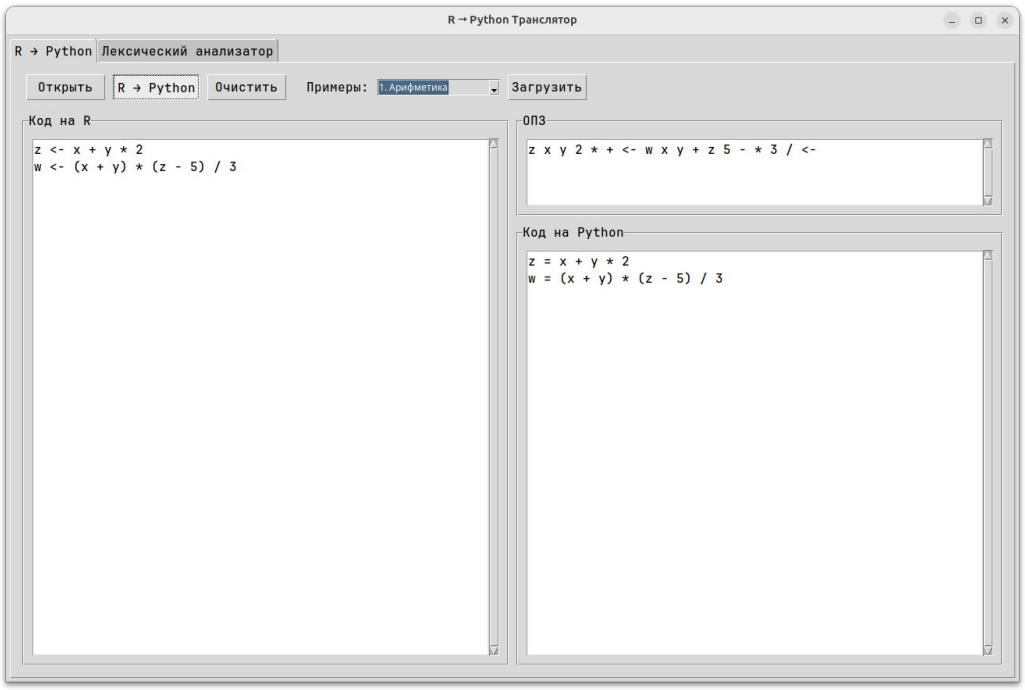
```
if x > 10:
    x = x - 1
else:
    x = 3
y = (a + 1 if a > b else b - 1)
```

Программа

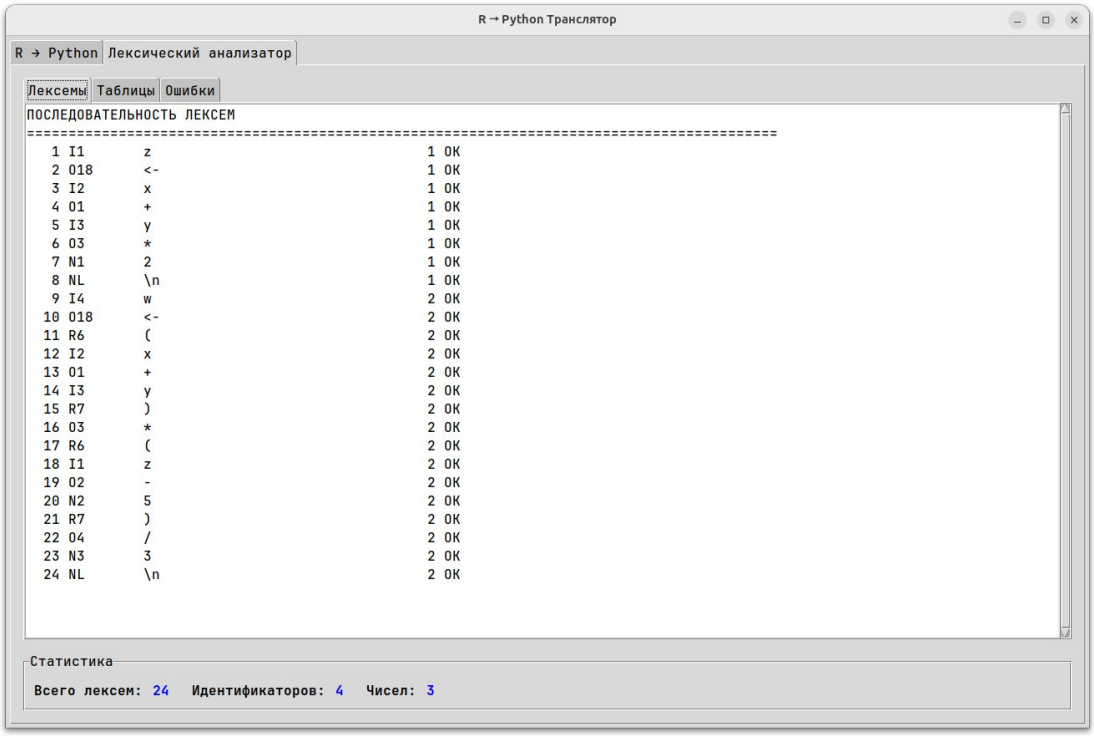
Программа представляет собой приложение с графическим интерфейсом. В левой части окна находится панель ввода исходного кода на языке R, а справа находятся результаты перевода в обратную польскую запись и код на выходном языке Python.



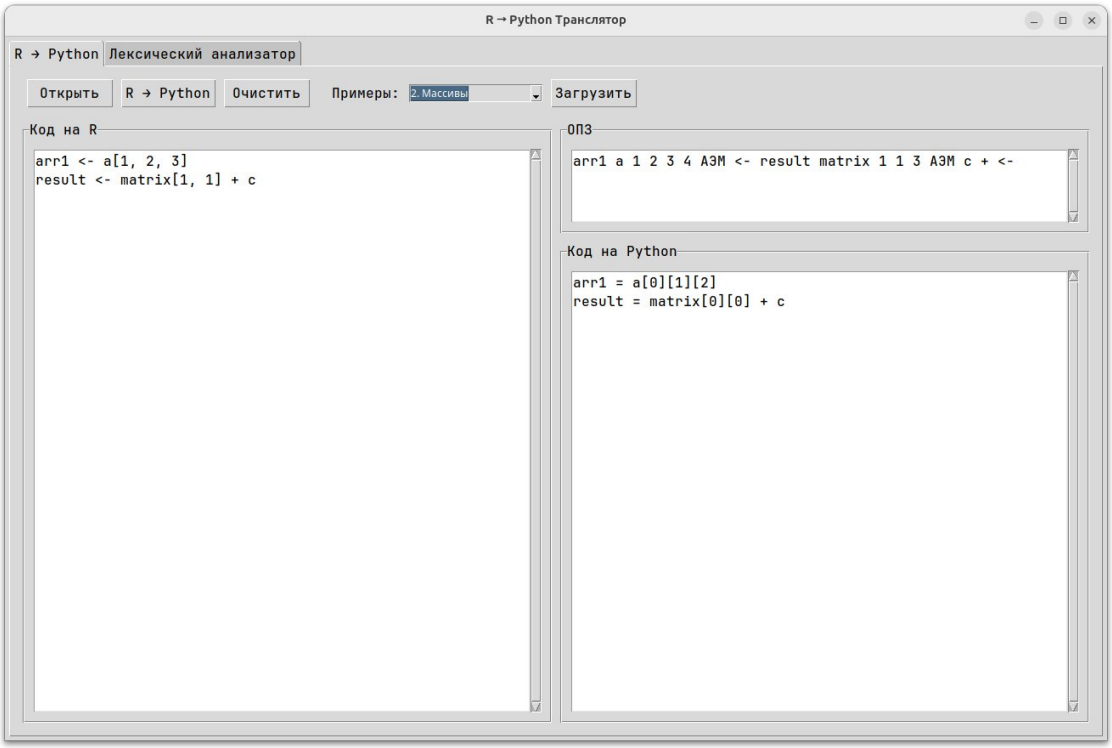
Программа поддерживает режим автоматического перевода в ОПЗ и код на Python.



Помимо автоперевода программа предоставляет пользователю возможность посмотреть на результаты лексического анализа входного кода на R.



Пользователь может выбрать какой-либо из примеров для демонстрации работы программы.



Структура компонентов программы

Класс PythonCodeGenerator

Данный класс реализует подход, основанный на интерпретации команд ОПЗ (УПЛ, БП, АЭМ и т.д.). Он эмулирует работу стековой вычислительной машины с генерацией кода (с использованием временных переменных).

1 Атрибуты состояния

Атрибут	Тип	Назначение
output_lines	list	Буфер строк результирующего Python-кода.
python_stack	list	Стек операндов. Хранит имена переменных, временные регистры _tN или готовые выражения.
indent_level	int	Текущий уровень отступа (для корректной вставки if, else, def).
temp_counter	int	Счетчик для генерации уникальных имен временных переменных (_t1, _t2, ...).
history	list	История состояний для пошаговой отладки генерации.
ifelse_temp	any	Флаг/хранилище состояния для обработки тернарного оператора ifelse.
current_step	int	Указатель текущего шага в истории для отладки.

2 Вспомогательные методы

Метод	Описание
_add_line(line)	Добавляет строку кода в output_lines с учетом текущего отступа (indent_level).
_save_state(action)	Сохраняет снимок (стек, код, отступы) в массив history. Используется для визуализации процесса трансляции.
_convert_value(token)	Преобразует литералы R в литералы Python: TRUE -> True, NULL -> None, Inf -> float('inf').

3 Семантические процедуры обработки ОПЗ

Каждый метод соответствует обработке конкретной лексемы ОПЗ, встречающейся во входном потоке.

Лексема ОПЗ	**Метод класса**	**Алгоритм работы и генерируемый код**
+, -, *, /, ^, >, <, &, |	_process_binary_op	1. Извлекает right и left из стека. 2. Преобразует оператор (^ -> **, & -> and). 3. Генерирует присваивание временной переменной: _tN = (left op right). 4. Помещает _tN обратно в стек.
:=, =, <-	_process_assignment	Извлекает из стека выражение (expr) и переменную (var). Генерирует строку: var = expr.
УПЛ	_process_conditional_jump	Извлекает метку и условие. Генерирует строку if cond: и увеличивает indent_level.
БП	_process_unconditional_jump	Извлекает метку. Уменьшает отступ и генерирует строку else:.
АЭМ	_process_array_access	1. Определяет размерность массива. 2. Извлекает имя и индексы. 3. **Ключевая особенность:** Применяет к каждому индексу операцию -1 (т.к. в R индексы с 1, в Python с 0). 4. Генерирует строку вида arr[idx1-1][idx2-1] и кладет в стек.
Ф	_process_function_call	1. Определяет число аргументов. 2. Извлекает имя функции и аргументы из стека. 3. **Обработка ifelse:** Если имя функции ifelse, генерируется тернарное Python-выражение: (true_val if cond else false_val). 4. **Обычный вызов:** Генерирует _tN = func(arg1, arg2) и кладет _tN в стек.
НП	_process_proc_begin	Извлекает имя процедуры. Генерирует def name():, увеличивает отступ.
КП	_process_proc_end	Уменьшает отступ и добавляет пустую строку для разделения функций.
МЦ	_process_while_condition	Извлекает условие. Генерирует while cond:, увеличивает отступ.

Полный текст разработанной программы:

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import re

from lexer import RLexer, LexemType
from polska import OPZConverter


class PythonCodeGenerator:
    def __init__(self):
        self.reset()

    def reset(self):
        self.output_lines = []
        self.python_stack = []
        self.indent_level = 0
        self.temp_counter = 0
        self.history = []
        self.current_token = ""
        self.current_step = 0
        self.label_positions = {}
        self.ifelse_temp = None

    def _add_line(self, line):
        indent = "    " * self.indent_level
        if line.strip():
            self.output_lines.append(indent + line)
        else:
            self.output_lines.append("")

    def _save_state(self, action):
        state = {
            'step': len(self.history),
            'action': action,
            'token': self.current_token,
            'stack': self.python_stack.copy(),
            'code': '\n'.join(self.output_lines),
            'indent': self.indent_level
        }
        self.history.append(state)

    def generate(self, opz_tokens_str):
        self.reset()
        tokens = opz_tokens_str.split()

        self._save_state("Начало генерации")

        i = 0
        while i < len(tokens):
            token = tokens[i]
            self.current_token = token

            if token.endswith(':'):
                i += 1
                continue

            if token in ('+', '-', '*', '/', '^', '%%', '%/%',
                         '<', '>', '⩽', '⩾', '⩵', '≠',
                         '&', '|', '||', '&&'):
                self._process_binary_op(token)

            elif token in (':=', '=', '←', '«←', '->', '->>'):
                self._process_assignment()

            elif token == 'УПЛ':
                self._process_conditional_jump(tokens, i)

            elif token == 'БП':
                self._process_unconditional_jump(tokens, i)

            elif token == 'Ф':
                self._process_function_call(tokens, i)
```

```
        elif token == 'АЭМ':
            self._process_array_access(tokens, i)

        elif token == 'НП':
            self._process_proc_begin()

        elif token == 'КП':
            self._process_proc_end()

        elif token == 'WHILE':
            pass

        elif token.startswith('МЦ'):
            self._process_while_condition(tokens, i)

        elif token == 'ifelse':
            pass

        else:
            self.python_stack.append(self._convert_value(token))
            self._save_state(f"Операнд: {token}")

        i += 1

    self._save_state("Завершение")
    return '\n'.join(self.output_lines), self.history

def _convert_value(self, token):
    if token == "TRUE":
        return "True"
    elif token == "FALSE":
        return "False"
    elif token in ("NULL", "NA"):
        return "None"
    elif token == "Inf":
        return "float('inf')"
    elif token == "NaN":
        return "float('nan')"
    else:
        return token

def _process_binary_op(self, op):
    if len(self.python_stack) < 2:
        return

    right = self.python_stack.pop()
    left = self.python_stack.pop()

    py_op = {
        '^': '**', '**': '**',
        '%%': '%',
        '%/%': '//',
        '&': 'and', '&&': 'and',
        '|': 'or', '||': 'or'
    }.get(op, op)

    self.temp_counter += 1
    temp_var = f"_t{self.temp_counter}"

    expr = f"{temp_var} = ({left} {py_op} {right})"
    self._add_line(expr)
    self.python_stack.append(temp_var)
    self._save_state(f"Операция: {op}")

def _process_assignment(self):
    if len(self.python_stack) < 2:
        return

    expr = self.python_stack.pop()
    var = self.python_stack.pop()

    self._add_line(f"{var} = {expr}")
    self._save_state(f"Присваивание: {var} = {expr}")

def _process_conditional_jump(self, tokens, pos):
    if len(self.python_stack) < 2:
```

```
        return

    label = self.python_stack.pop()
    cond = self.python_stack.pop()

    if self.ifelse_temp is not None:
        self.ifelse_cond = cond
        self.ifelse_label1 = label
    else:
        self._add_line(f"if {cond}:")
        self.indent_level += 1
    self._save_state(f"УПЛ: if {cond}:")

def _process_unconditional_jump(self, tokens, pos):
    if len(self.python_stack) < 1:
        return

    label = self.python_stack.pop()

    if self.ifelse_temp is not None:
        self.ifelse_label2 = label
    else:
        if self.indent_level > 0:
            self.indent_level -= 1
        self._add_line(f"else:")
        self.indent_level += 1
    self._save_state(f"БП: goto {label}")

def _process_function_call(self, tokens, pos):
    arg_count = 2
    if pos > 0 and tokens[pos-1].isdigit():
        arg_count = int(tokens[pos-1])

    args = []
    for _ in range(arg_count):
        if self.python_stack:
            args.insert(0, self.python_stack.pop())

    if not args:
        return

    func_name = args[0]
    func_args = args[1:]

    if func_name == 'ifelse':
        if len(func_args) == 3:
            cond = func_args[0]
            true_val = func_args[1]
            false_val = func_args[2]
            expr = f"({true_val} if {cond} else {false_val})"
            self.python_stack.append(expr)
            self._save_state(f"ifelse: {cond}")
            return

    call_str = f"{func_name}({', '.join(func_args)})"

    self.temp_counter += 1
    temp_var = f"_t{self.temp_counter}"
    self._add_line(f"{temp_var} = {call_str}")
    self.python_stack.append(temp_var)
    self._save_state(f"Функция: {func_name}")

def _process_array_access(self, tokens, pos):
    dim_count = 2
    if pos > 0 and tokens[pos-1].isdigit():
        dim_count = int(tokens[pos-1])

    items = []
    for _ in range(dim_count):
        if self.python_stack:
            items.insert(0, self.python_stack.pop())

    if len(items) < 2:
        return

    arr_name = items[0]
    indices = items[1:]
```

```
        py_indices = []
        for idx in indices:
            if idx.isdigit():
                py_indices.append(str(int(idx) - 1))
            elif idx.startswith('-') and idx[1:].isdigit():
                py_indices.append(str(int(idx) - 1))
            else:
                py_indices.append(f"({idx} - 1)")

        access_str = arr_name
        for py_idx in py_indices:
            access_str += f"[{py_idx}]"

        self.python_stack.append(access_str)
        self._save_state(f"Массив: {arr_name}")

    def _process_proc_begin(self):
        if not self.python_stack:
            return

        name = self.python_stack.pop()
        self._add_line(f"def {name}():")
        self.indent_level += 1
        self._save_state(f"Начало функции: {name}")

    def _process_proc_end(self):
        if self.indent_level > 0:
            self.indent_level -= 1
        self._add_line("")
        self._save_state("Конец функции")

    def _process_while_condition(self, tokens, pos):
        if len(self.python_stack) < 1:
            return

        cond = self.python_stack.pop()

        label = tokens[pos] if pos < len(tokens) else "МЦ1"

        self._add_line(f"while {cond}:")
        self.indent_level += 1
        self._save_state(f"Цикл while: {cond}")

    def get_current_state(self):
        if 0 ≤ self.current_step < len(self.history):
            return self.history[self.current_step]
        return None

    def step_forward(self):
        if self.current_step < len(self.history) - 1:
            self.current_step += 1
            return self.history[self.current_step]
        return None

    def step_backward(self):
        if self.current_step > 0:
            self.current_step -= 1
            return self.history[self.current_step]
        return None

    def reset_to_start(self):
        self.current_step = 0
        return self.history[0] if self.history else None

    def go_to_end(self):
        self.current_step = len(self.history) - 1
        return self.history[-1] if self.history else None


class DirectPythonGenerator:
    def __init__(self):
        self.reset()

    def reset(self):
        self.output_lines = []
        self.indent_level = 0
```

```
def _add_line(self, line):
    indent = "    " * self.indent_level
    if line.strip():
        self.output_lines.append(indent + line)
    else:
        self.output_lines.append("")

def generate_from_tokens(self, tokens):
    self.reset()

    significant = [t for t in tokens if t.lex_type.name != 'COMMENT']

    i = 0
    while i < len(significant):
        token = significant[i]

        if token.value in ('←', '⇐', '=', ':='):
            i = self._handle_assignment(significant, i)
        elif token.value == 'if' and not self._is_ifelse(significant, i):
            i = self._handle_if(significant, i)
        elif token.value == 'while':
            i = self._handle_while(significant, i)
        elif token.value == 'function':
            i = self._handle_function(significant, i)
        elif token.value == 'return':
            i = self._handle_return(significant, i)
        elif token.value == '{':
            i += 1
        elif token.value == '}':
            i += 1
        else:
            i += 1

    result = '\n'.join(self.output_lines)
    result = self._apply_brutal_fixes(result)
    return result

def _apply_brutal_fixes(self, code):
    code = re.sub(r'^(\s*)(\w+)\s*=\s*function\s*\(\s*\)\s*{', r'\1def \2():', code, flags=re.MULTILINE)
    code = re.sub(r'^(\s*)(\w+)\s*=\s*function\s*\(([^)]*)\)\s*{', r'\1def \2(\3):', code, flags=re.MULTILINE)

    code = re.sub(r'def\s+(\w+)\s*\(\s*\)\s*{', r'def \1():', code)
    code = re.sub(r'def\s+(\w+)\s*\(([^)]*)\)\s*{', r'def \1(\2):', code)

    lines = code.split('\n')
    filtered_lines = []
    for line in lines:
        stripped = line.strip()
        if stripped == '{' or stripped == '}':
            continue
        filtered_lines.append(line)
    code = '\n'.join(filtered_lines)

    def fix_ifelse(match):
        cond = match.group(1).strip()
        true_val = match.group(2).strip()
        false_val = match.group(3).strip()
        return f"({true_val} if {cond} else {false_val})"

    code = re.sub(r'ifelse\s*\(\s*([^,]+)\s*,\s*([^,]+)\s*,\s*([^)]+)\s*\)', fix_ifelse, code)

    def fix_array(match):
        arr = match.group(1)
        indices = match.group(2)
        idx_list = [i.strip() for i in indices.split(',')]
        py_indices = []
        for idx in idx_list:
            if idx.isdigit():
                py_indices.append(str(int(idx) - 1))
            elif idx.lstrip('-').isdigit():
                py_indices.append(str(int(idx) - 1))
            else:
                py_indices.append(f"({idx} - 1)")
        return f"{arr}[{']['.join(py_indices)}]"

    code = re.sub(r'(\w+)\s*\[\s*([^\]]+)\s*\]', fix_array, code)
```

```
        code = re.sub(r'if\s*\(\s*([^{]+)\s*\)\s*{', r'if \1:', code)

        code = re.sub(r'while\s*\(\s*([^{]+)\s*\)\s*{', r'while \1:', code)

        code = re.sub(r'}\s*else\s*{', r'else:', code)

        code = code.replace('( ', '(').replace(' )', ')')
        code = code.replace('[ ', '[').replace(' ]', ']')
        code = code.replace(' , ', ', ')

        lines = code.split('\n')
        fixed_lines = []
        in_function = False
        function_indent = 0

        for line in lines:
            stripped = line.strip()

            if stripped.startswith('def ') and stripped.endswith(':'):
                in_function = True
                fixed_lines.append(line)
                continue

            if in_function and not stripped:
                in_function = False
                fixed_lines.append(line)
                continue

            if in_function and stripped:
                if re.match(r'^(def |if |while |else:)', stripped):
                    in_function = False
                    fixed_lines.append(line)
                else:
                    fixed_lines.append("    " + line)
            else:
                fixed_lines.append(line)

        code = '\n'.join(fixed_lines)

        return code

    def _is_ifelse(self, tokens, pos):
        for j in range(pos, min(pos + 10, len(tokens))):
            if tokens[j].value == 'ifelse':
                return True
        return False

    def _convert_token(self, token):
        val = token.value
        if val == 'TRUE': return 'True'
        if val == 'FALSE': return 'False'
        if val in ('NULL', 'NA'): return 'None'
        if val == 'Inf': return "float('inf')"
        if val == 'NaN': return "float('nan')"
        if val in ('^', '**'): return '**'
        if val == '%%': return '%'
        if val == '%/%': return '//'
        if val in ('&', '&&'): return 'and'
        if val in ('|', '||'): return 'or'
        if val == '!': return 'not'
        return val

    def _parse_expression(self, tokens, start, stop_tokens=None):
        if stop_tokens is None:
            stop_tokens = {'\n', ';', '}', '{', 'else', ')', ']', ','}

        parts = []
        i = start

        while i < len(tokens):
            t = tokens[i]

            if t.value in stop_tokens:
                break

            if t.lex_type == LexemType.IDENTIFIER and i + 1 < len(tokens) and tokens[i+1].value == '(':
```

```
            func_name = t.value
            i += 2

            args = []
            while i < len(tokens) and tokens[i].value ≠ ')':
                if tokens[i].value == ',':
                    i += 1
                    continue
                arg, i = self._parse_expression(tokens, i, {',', ')'})
                if arg is not None:
                    args.append(arg)

            parts.append(f"{func_name}({', '.join(args)})")

            if i < len(tokens) and tokens[i].value == ')':
                i += 1
            continue

        if t.lex_type == LexemType.IDENTIFIER and i + 1 < len(tokens) and tokens[i+1].value == '[':
            arr_name = t.value
            i += 2

            indices = []
            while i < len(tokens) and tokens[i].value ≠ ']':
                if tokens[i].value == ',':
                    i += 1
                    continue
                idx, i = self._parse_expression(tokens, i, {',', ']'})
                if idx is not None:
                    indices.append(str(idx).strip())

            parts.append(f"{arr_name}[{', '.join(indices)}]")

            if i < len(tokens) and tokens[i].value == ']':
                i += 1
            continue

        if t.value == '(':
            sub_expr, i = self._parse_expression(tokens, i + 1, {')'})
            parts.append(f"({sub_expr})")
            if i < len(tokens) and tokens[i].value == ')':
                i += 1
            continue

        parts.append(self._convert_token(t))
        i += 1

    return ' '.join(parts), i

def _handle_assignment(self, tokens, pos):
    left_tokens = []
    i = pos - 1
    while i ≥ 0:
        t = tokens[i]
        if t.value in (';', '\n', '{', '}', 'if', 'else', 'while', 'function', 'return', '←', '=', ':='):
            break
        left_tokens.insert(0, t)
        i -= 1

    left, _ = self._parse_expression(left_tokens, 0)
    right, end = self._parse_expression(tokens, pos + 1, {'\n', ';', '}', 'else'})

    self._add_line(f"{left} = {right}")
    return end

def _handle_if(self, tokens, pos):
    i = pos + 1
    if i < len(tokens) and tokens[i].value == '(':
        i += 1

    cond, i = self._parse_expression(tokens, i, {')', '{', '\n'})
    cond = cond.strip()
    if cond.startswith('(') and cond.endswith(')'):
        cond = cond[1:-1]

    self._add_line(f"if {cond}:")
    self.indent_level += 1
```

```
        if i < len(tokens) and tokens[i].value == ')':
            i += 1
        i = self._parse_block(tokens, i)

        while i < len(tokens) and tokens[i].value in ('\n', ' '):
            i += 1

        if i < len(tokens) and tokens[i].value == 'else':
            self.indent_level -= 1
            self._add_line("else:")
            self.indent_level += 1
            i = self._parse_block(tokens, i + 1)

        self.indent_level -= 1
        return i

    def _handle_while(self, tokens, pos):
        i = pos + 1
        if i < len(tokens) and tokens[i].value == '(':
            i += 1

        cond, i = self._parse_expression(tokens, i, {')', '{', '\n'})
        cond = cond.strip()
        if cond.startswith('(') and cond.endswith(')'):
            cond = cond[1:-1]

        self._add_line(f"while {cond}:")
        self.indent_level += 1

        if i < len(tokens) and tokens[i].value == ')':
            i += 1
        i = self._parse_block(tokens, i)
        self.indent_level -= 1

        return i

    def _handle_function(self, tokens, pos):
        func_name = "func"
        i = pos - 1
        while i ≥ 0 and tokens[i].value in ('←', '=', ' ', '\n'):
            i -= 1
        if i ≥ 0 and tokens[i].lex_type == LexemType.IDENTIFIER:
            func_name = tokens[i].value

        params = []
        i = pos + 1
        if i < len(tokens) and tokens[i].value == '(':
            i += 1
            while i < len(tokens) and tokens[i].value ≠ ')':
                if tokens[i].value == ',':
                    i += 1
                    continue
                param, i = self._parse_expression(tokens, i, {',', ')'})
                if param:
                    params.append(param.strip())
            if i < len(tokens) and tokens[i].value == ')':
                i += 1

        self._add_line(f"def {func_name}({', '.join(params)}):")
        self.indent_level += 1

        i = self._parse_block(tokens, i)
        self.indent_level -= 1
        self._add_line("")

        return i

    def _handle_return(self, tokens, pos):
        expr, i = self._parse_expression(tokens, pos + 1, {'\n', ';', '}'})
        self._add_line(f"return {expr}")
        return i

    def _parse_block(self, tokens, start):
        i = start
        while i < len(tokens) and tokens[i].value in (' ', '\n'):
            i += 1
```

```
        if i < len(tokens) and tokens[i].value == '{':
            i += 1
            while i < len(tokens) and tokens[i].value != '}':
                t = tokens[i]
                if t.value in ('←', '⇐', '=', ':='):
                    i = self._handle_assignment(tokens, i)
                elif t.value == 'if' and not self._is_ifelse(tokens, i):
                    i = self._handle_if(tokens, i)
                elif t.value == 'while':
                    i = self._handle_while(tokens, i)
                elif t.value == 'return':
                    i = self._handle_return(tokens, i)
                elif t.value == 'function':
                    i = self._handle_function(tokens, i)
                else:
                    i += 1
            return i + 1
        else:
            stmt, i = self._parse_expression(tokens, i, {'\n', ';', '}', 'else'})
            if stmt:
                self._add_line(stmt)
            return i


class SharedData:
    def __init__(self):
        self.tokens = []
        self.errors = []
        self.identifiers = {}
        self.numbers = {}
        self.strings = {}
        self.comments = {}
        self.keywords = {}
        self.delimiters = {}
        self.operations = {}


class LexicalAnalyzerGUI:
    def __init__(self, parent, shared, default_font):
        self.parent = parent
        self.shared = shared
        self.default_font = default_font
        self.setup_ui()

    def setup_ui(self):
        main = ttk.Frame(self.parent, padding="15")
        main.pack(fill=tk.BOTH, expand=True)
        main.columnconfigure(0, weight=1)
        main.rowconfigure(0, weight=1)

        notebook = ttk.Notebook(main)
        notebook.grid(row=0, column=0, sticky="nsew")

        tf = ttk.Frame(notebook)
        notebook.add(tf, text="Лексемы")
        tf.columnconfigure(0, weight=1)
        tf.rowconfigure(0, weight=1)
        self.tokens_text = scrolledtext.ScrolledText(tf, font=self.default_font)
        self.tokens_text.grid(row=0, column=0, sticky="nsew")

        tf2 = ttk.Frame(notebook)
        notebook.add(tf2, text="Таблицы")
        tf2.columnconfigure(0, weight=1)
        tf2.rowconfigure(0, weight=1)
        self.tables_text = scrolledtext.ScrolledText(tf2, font=self.default_font)
        self.tables_text.grid(row=0, column=0, sticky="nsew")

        ef = ttk.Frame(notebook)
        notebook.add(ef, text="Ошибки")
        ef.columnconfigure(0, weight=1)
        ef.rowconfigure(0, weight=1)
        self.errors_text = scrolledtext.ScrolledText(ef, font=self.default_font, bg='#fff0f0', fg='red')
        self.errors_text.grid(row=0, column=0, sticky="nsew")

        sf = ttk.LabelFrame(main, text="Статистика", padding="10")
        sf.grid(row=1, column=0, sticky="ew", pady=(15, 0))
```

```
        self.stats_labels = {}
        items = [("Всего лексем:", 0), ("Идентификаторов:", 2), ("Чисел:", 4)]

        bold_font = (self.default_font[0], self.default_font[1], 'bold')
        for i, (label, col) in enumerate(items):
            ttk.Label(sf, text=label, font=bold_font).grid(row=0, column=col, sticky='w', padx=(20 if i > 0 else
0, 5))
            self.stats_labels[label] = ttk.Label(sf, text="0", foreground="blue", font=bold_font)
            self.stats_labels[label].grid(row=0, column=col + 1, sticky='w', padx=5)

    def update(self):
        if not self.shared.tokens: return
        self._update_tokens()
        self._update_tables()
        self._update_errors()
        self._update_stats()

    def _update_tokens(self):
        self.tokens_text.delete(1.0, tk.END)
        self.tokens_text.insert(1.0, "ПОСЛЕДОВАТЕЛЬНОСТЬ ЛЕКСЕМ\n" + "=" * 90 + "\n")
        for i, t in enumerate(self.shared.tokens[:200], 1):
            status = "ERR" if t.lex_type == LexemType.ERROR else "OK"
            val = t.value.replace('\n', '\\n')
            self.tokens_text.insert(tk.END, f"{i:4d} {t.code:8s} {val[:30]:30s} {t.line:4d} {status:5s}\n")

    def _update_tables(self):
        self.tables_text.delete(1.0, tk.END)
        out = ["ТАБЛИЦЫ ЛЕКСЕМ"]
        out.append("\nКЛЮЧЕВЫЕ СЛОВА:")
        for w, i in sorted(self.shared.keywords.items(), key=lambda x: x[1])[:20]:
            out.append(f"  W{i:4d} : {w}")
        out.append("\nИДЕНТИФИКАТОРЫ:")
        for n, i in sorted(self.shared.identifiers.items(), key=lambda x: x[1])[:20]:
            out.append(f"  I{i:4d} : {n}")
        self.tables_text.insert(1.0, '\n'.join(out))

    def _update_errors(self):
        self.errors_text.delete(1.0, tk.END)
        if self.shared.errors:
            for e in self.shared.errors[:50]:
                self.errors_text.insert(tk.END, f"{e}\n")
        else:
            self.errors_text.insert(1.0, "Ошибок нет")

    def _update_stats(self):
        stats = {
            "Всего лексем:": len(self.shared.tokens),
            "Идентификаторов:": len(self.shared.identifiers),
            "Чисел:": len(self.shared.numbers),
        }
        for label, val in stats.items():
            if label in self.stats_labels:
                self.stats_labels[label].config(text=str(val))


class PythonGeneratorGUI:
    def __init__(self, parent, shared, lexer_app, default_font):
        self.parent = parent
        self.shared = shared
        self.lexer_app = lexer_app
        self.default_font = default_font
        self.generator = DirectPythonGenerator()
        self.opz_converter = OPZConverter()
        self.setup_ui()

    def setup_ui(self):
        main = ttk.Frame(self.parent, padding="15")
        main.pack(fill=tk.BOTH, expand=True)
        main.columnconfigure(0, weight=1)
        main.columnconfigure(1, weight=1)
        main.rowconfigure(1, weight=1)

        toolbar = ttk.Frame(main)
        toolbar.grid(row=0, column=0, columnspan=2, sticky="ew", pady=(0, 15))
        ttk.Button(toolbar, text="Открыть", command=self.open_file).pack(side=tk.LEFT, padx=5)
        ttk.Button(toolbar, text="R → Python", command=self.auto_convert).pack(side=tk.LEFT, padx=5)
```

```
        ttk.Button(toolbar, text="Очистить", command=self.clear_all).pack(side=tk.LEFT, padx=5)

        ttk.Label(toolbar, text="Примеры:").pack(side=tk.LEFT, padx=(20, 5))
        self.example_var = tk.StringVar()
        self.example_combo = ttk.Combobox(
            toolbar,
            textvariable=self.example_var,
            values=["1. Арифметика", "2. Массивы", "3. Функции", "4. ifelse", "5. if-else", "6. while", "7.
function", "8. Всё вместе"],
            state="readonly",
            width=20
        )
        self.example_combo.pack(side=tk.LEFT, padx=5)
        ttk.Button(toolbar, text="Загрузить", command=self.load_selected_example).pack(side=tk.LEFT, padx=5)

        left = ttk.LabelFrame(main, text="Код на R", padding="10")
        left.grid(row=1, column=0, sticky="nsew", padx=(0, 10))
        left.columnconfigure(0, weight=1)
        left.rowconfigure(0, weight=1)
        self.code_text = scrolledtext.ScrolledText(left, font=self.default_font)
        self.code_text.grid(row=0, column=0, sticky="nsew")

        right = ttk.Frame(main)
        right.grid(row=1, column=1, sticky="nsew")
        right.columnconfigure(0, weight=1)
        right.rowconfigure(0, weight=1)
        right.rowconfigure(1, weight=2)

        opz_frame = ttk.LabelFrame(right, text="ОПЗ", padding="10")
        opz_frame.grid(row=0, column=0, sticky="nsew", pady=(0, 10))
        opz_frame.columnconfigure(0, weight=1)
        opz_frame.rowconfigure(0, weight=1)
        self.opz_text = scrolledtext.ScrolledText(opz_frame, font=self.default_font, height=4)
        self.opz_text.grid(row=0, column=0, sticky="nsew")

        py_frame = ttk.LabelFrame(right, text="Код на Python", padding="10")
        py_frame.grid(row=1, column=0, sticky="nsew")
        py_frame.columnconfigure(0, weight=1)
        py_frame.rowconfigure(0, weight=1)
        self.python_text = scrolledtext.ScrolledText(py_frame, font=self.default_font)
        self.python_text.grid(row=0, column=0, sticky="nsew")

        self.example_var.set("1. Арифметика")
        self.load_selected_example()

    def get_examples(self):
        return {
            "1. Арифметика": "z ← x + y * 2\nw ← (x + y) * (z - 5) / 3",
            "2. Массивы": "arr1 ← a[1, 2, 3]\nresult ← matrix[1, 1] + c",
            "3. Функции": "x ← sum(1, 2, 3)\nresult ← f(2, y, x) / 123",
            "4. ifelse": "z ← ifelse(x > y, x + 1, y - 1)",
            "5. if-else": "if (x > 10) {\n    x ← x - 1\n} else {\n    x ← 3\n}",
            "6. while": "while (i < 10) {\n    sum ← sum + i\n    i ← i + 1\n}",
            "7. function": "f ← function() {\n    z ← x + y\n    t ← z + 1\n}",
            "8. Всё вместе": """g ← function() {
    x ← x + 1
    y ← y[5]
    t ← ifelse(x > 1, y, z)
    u ← matrix[1, 1]
}
if (a > b) {
    x ← h(x, 6)
} else {
    x ← 2
}
while (x > 0) {
    x ← f(x - 1, y)
    y ← y + 1
}"""
        }

    def load_selected_example(self):
        selected = self.example_var.get()
        examples = self.get_examples()
        if selected in examples:
            self.code_text.delete(1.0, tk.END)
            self.code_text.insert(1.0, examples[selected])
```

```
    def open_file(self):
        fn = filedialog.askopenfilename(filetypes=[("R files", "*.r"), ("All", "*.*")])
        if fn:
            with open(fn, 'r', encoding='utf-8') as f:
                self.code_text.delete(1.0, tk.END)
                self.code_text.insert(1.0, f.read())

    def run_lexical(self, code):
        lexer = RLexer()
        self.shared.tokens = lexer.tokenize(code)
        self.shared.errors = lexer.errors.copy()
        self.shared.identifiers = lexer.identifiers.copy()
        self.shared.numbers = lexer.numbers.copy()
        self.shared.strings = lexer.strings.copy()
        self.shared.comments = lexer.comments.copy()
        self.shared.keywords = lexer.keywords.copy()
        self.shared.delimiters = lexer.delimiters.copy()
        self.shared.operations = lexer.operations.copy()
        self.lexer_app.update()
        return self.shared.tokens

    def auto_convert(self):
        code = self.code_text.get(1.0, tk.END)
        if not code.strip():
            return

        try:
            tokens = self.run_lexical(code)

            significant = [t for t in tokens if t.lex_type != LexemType.COMMENT]
            opz_str, _ = self.opz_converter.convert_expression(significant)
            self.opz_text.delete(1.0, tk.END)
            self.opz_text.insert(1.0, opz_str)

            self.generator.reset()
            python_code = self.generator.generate_from_tokens(tokens)
            self.python_text.delete(1.0, tk.END)
            self.python_text.insert(1.0, python_code)

        except Exception as e:
            messagebox.showerror("Ошибка", str(e))
            import traceback
            traceback.print_exc()

    def clear_all(self):
        self.code_text.delete(1.0, tk.END)
        self.opz_text.delete(1.0, tk.END)
        self.python_text.delete(1.0, tk.END)
        self.shared.tokens = []
        self.shared.errors = []
        self.lexer_app.update()


class MainApplication:
    def __init__(self, root):
        self.root = root
        self.root.title("R → Python Транслятор")
        self.root.geometry("1600x900")

        self.default_font = ('JetBrains Mono', 12)
        style = ttk.Style()
        style.configure('.', font=self.default_font)

        self.shared = SharedData()

        self.notebook = ttk.Notebook(root)
        self.notebook.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

        self.python_frame = ttk.Frame(self.notebook)
        self.notebook.add(self.python_frame, text="R → Python")

        self.lexer_frame = ttk.Frame(self.notebook)
        self.notebook.add(self.lexer_frame, text="Лексический анализатор")
        self.lexer_app = LexicalAnalyzerGUI(self.lexer_frame, self.shared, self.default_font)

        self.python_app = PythonGeneratorGUI(self.python_frame, self.shared, self.lexer_app, self.default_font)
```

```
if __name__ == "__main__":
    root = tk.Tk()
    app = MainApplication(root)
    root.mainloop()
```

Вывод

Разработанная программа успешно реализует функции генератора кода, преобразуя промежуточное представление (ОПЗ) языка R в код на языке Python. Программа учитывает различия языков: 1-базированную индексацию R против 0-базированной в Python, синтаксис блоков с помощью фигурных скобок против отступов, а также наличие специфических векторных функций.